

RESEARCH DOCUMENT V1.0

UI Architecture for the Influence Accelerator

A comprehensive analysis of design patterns, interaction models, and architectural approaches for building the IA Psychological Engineering Workstation and Creator Economy Accelerator — grounded in your R&D corpus, first principles, and the best practices of Apple, Figma, Linear, and frontier AI tools.

Compiled [August 4, 2026](#) · [Rithwik Gopal](#) · [40+ Sources](#)

Table of Contents

1. The System You Are Building (Synthesis of All Docs)
2. The Core UI Problem: Layered Abstraction
3. Approach 1: Hierarchical Drill-Down (Current Direction)
4. Approach 2: Canvas + Inspector (Figma / Logic Pro Model)
5. Approach 3: Conversational Shell + Contextual Panels (Cursor / Arc Model)
6. Approach 4: Graph-First Workstation (Unreal / ComfyUI Model)
7. Approach 5: Hybrid — The Recommended Architecture
8. Comparison Matrix: All 5 Approaches
9. Design Principles: Lessons from the Best
10. Key People & Resources
11. The Glassmorphism Question
12. Psychological Engineering in the UI Itself
13. Recommendations & Next Steps

The System You Are Building

Before exploring UI approaches, we must first ground the discussion in **what the system actually is**. After reading every document in your R&D folder — including the_answer.md, user_vision_and_execution_plan.md, implementation_plan.md, project_status_and_roadmap.md, the agent specs, mermaid diagrams, Avatar_Internal.md, CLAUDE.md, the DSPy learning sessions, and the Conditions Deck — here is the synthesized picture:

The Influence Accelerator is a Neurosymbolic Content Engineering OS.

It has two operating modes that share the same underlying engine but present different levels of abstraction to the user:

PEW — Psychological Engineering Workstation

The raw engine. First-principles psychology (16 levels of Human Sociality). Node-based constraint harness. Symbolic + Neural layers. Domain-agnostic. Works for any medium: social media, sales letters, speeches, product positioning.

CEA — Creator Economy Accelerator

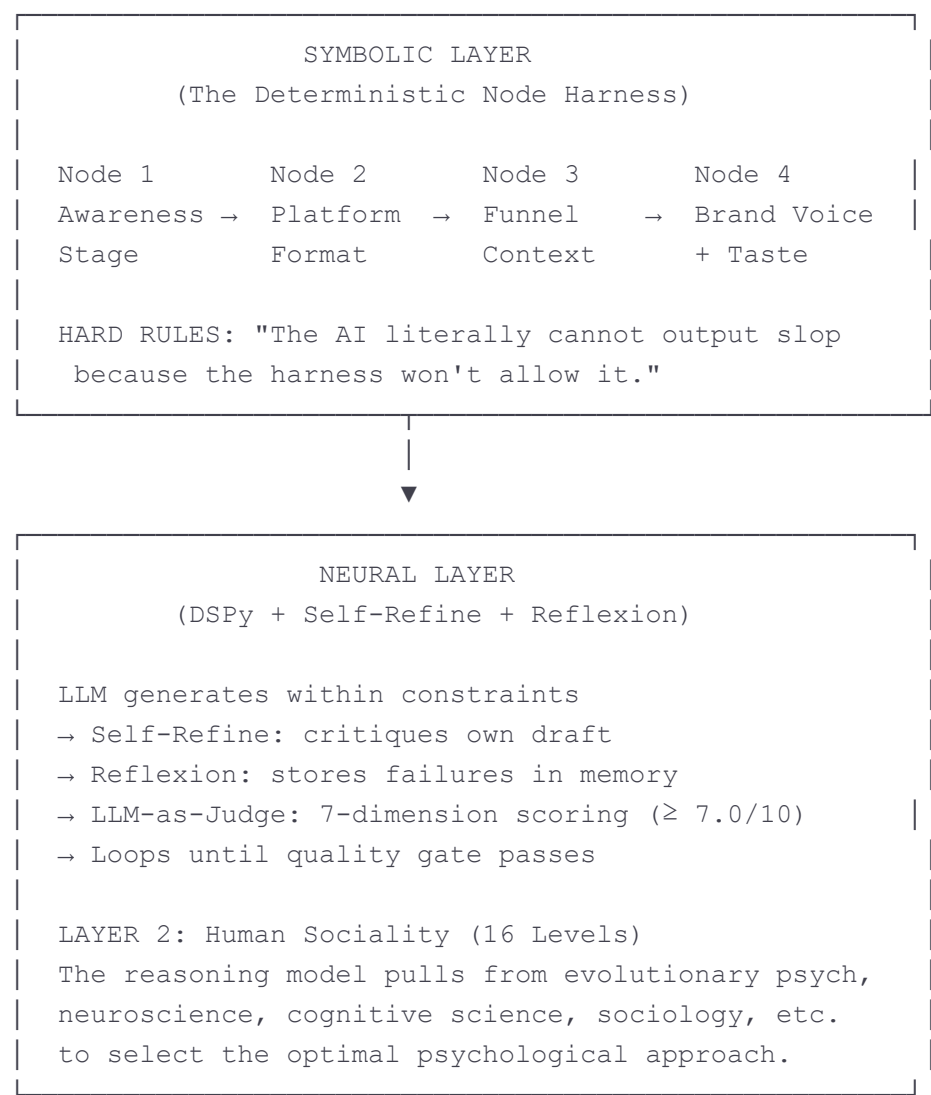
A fine-tuned wrapper on top of PEW. Specifically optimized for social media distribution, reach, and creator monetization. Uses the same engine but with platform-specific constraints (IG Reels, YouTube, X, LinkedIn) and creator-economy terminology (Avatar, Content, Conversion Chain).

The Three Pillars (Shared by Both Modes)

Regardless of whether you call them Human/Stimulus/Action (PEW) or Avatar/Content/Conversion Chain (CEA), the system operates on three fundamental pillars:

THE SUBJECT	THE PAYLOAD	THE PATH
Human / Avatar	Stimulus / Content	Action / Conversion Chain
28 psychographic data points.	AI-generated copy that flips	The psychological journey
The conditions deck. The	incorrect beliefs. Passes	map from first contact to
belief architecture. The	through 5-node constraint	conversion. Each node = a
internal + market reality.	harness + Self-Refine + LLM-	state transition in the subject's
	as-Judge quality gate.	psychology.

The Neurosymbolic Stack (Under the Hood)



The Core UI Problem: Layered Abstraction

The fundamental challenge is: **How do you expose a system this deep through a visual interface without overwhelming the user or dumbing it down?**

You are dealing with at least 4 layers of depth:

Layer	What It Contains	Interaction Type
1. Mode Selection	Chat / CEA / PEW	Simple toggle
2. Pillar Selection	Avatar / Content / Conversion Chain (or Human / Stimulus / Action)	3 big blocks
3. Pillar Editor	28 psychographic fields, belief architectures, node chains, copy outputs	Deep data entry + canvas
4. Engine Config	DSPy nodes, quality thresholds, Layer 2 reasoning weights, model selection	Advanced settings (hidden by default)

The question is: **What is the optimal interaction model for navigating between these layers?**

Below are 5 distinct approaches, each modeled after a real-world system that has solved a similar problem. Each has trade-offs.

Approach 1: Hierarchical Drill-Down

The iOS Settings Model

Current Direction

Inspired by: Apple iOS Settings, current IA UI, Linear's nested views

Concept: Strict parent → child navigation. Each click drills you one level deeper. Back button returns you one level up. No side-by-side views.

How it maps to IA:

1. Home Screen: Chat + Mode Toggle (Chat / CEA / PEW)
2. Click CEA → See 3 big blocks (Avatar, Content, Conversion Chain)
3. Click Avatar → Full-screen editor with 28 fields
4. Back → Returns to 3-block overview

☒ Strengths:

Dead simple mental model. Zero cognitive overload. Clean navigation breadcrumb. Each screen has one purpose. Mobile-friendly. Easy to implement.

Weaknesses:

You lose context of the other two pillars when editing one. The system's power comes from the *relationship* between pillars (Avatar constraints → shape Stimulus → inform Action Chain). A strictly hierarchical UI hides these relationships. You're always in one tunnel. This is the "iOS Settings" problem — great for linear config, bad for systems thinking.

Best for: Initial onboarding / data entry phase. Bad for the active "engineering" phase where you're iterating across all three pillars simultaneously.

Approach 2: Canvas + Inspector

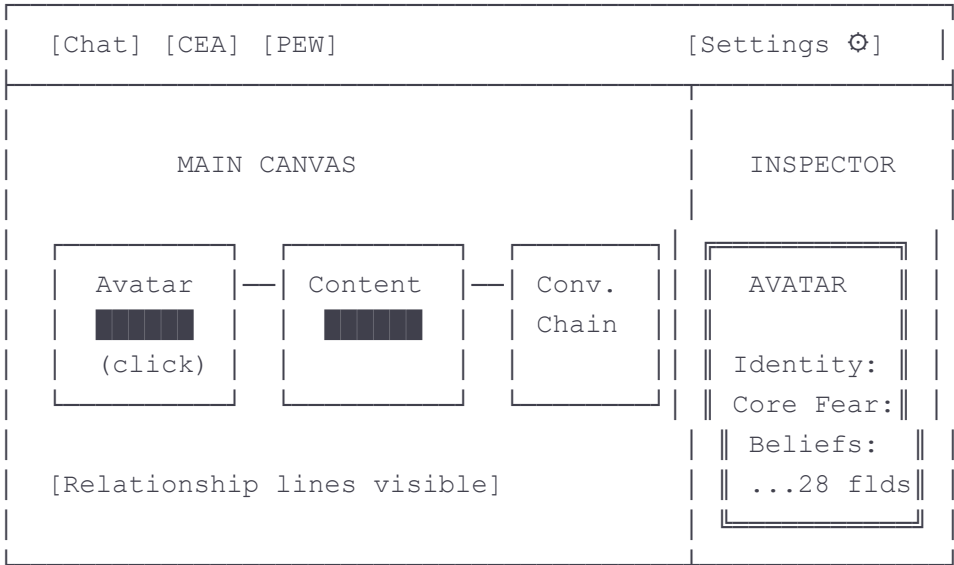
The Figma / Logic Pro Model

Recommended for CEA

Inspired by: Figma, Apple Logic Pro X, Adobe Premiere, Notion databases

Concept: A persistent canvas (the main workspace) with a contextual Inspector panel on the right that changes based on what you've selected.

How it maps to IA:



- The 3 pillars are always visible as blocks/cards on the canvas.
- Click any pillar → the **Inspector panel** on the right populates with that pillar's detailed editor (forms, fields, node chains).

- You can **see the relationships** between pillars while editing one.
- The Inspector can be collapsed for a wider canvas view.

Strengths:

Maintains system awareness at all times. You never lose context. The relationship between Avatar → Content → Conversion Chain is always visible. Figma users, designers, and system thinkers feel at home. Scales beautifully as you add more components.

Weaknesses:

More complex to implement. Requires responsive split-pane logic. The Inspector can feel cramped on smaller screens. Needs clear visual hierarchy to avoid overwhelming the canvas.

Best for: The "engineering" phase where you're actively iterating. This is how Figma, Logic Pro, and Premiere handle deep creative tools — always-visible context + deep editing on demand.

Approach 3: Conversational Shell + Contextual Panels

The Cursor / Arc Browser Model

Most "AI-Native"

Inspired by: Cursor IDE Composer, Arc Browser Spaces, Claude Artifacts, Vercel v0

Concept: Chat is the primary interface. But when the AI generates structured output (Avatar data, copy drafts, conversion chains), it materializes as interactive panels alongside the chat.

How it maps to IA:

[Chat] [CEA] [PEW]	
CHAT THREAD	GENERATED ARTIFACT
You: "Set up avatar for burned-out agency owners age 28-35"	AVATAR PANEL
AI: "I've populated the avatar with 28 data points based on your input + market research..."	Identity: Agency Owner
	Core Fear: Irrelevance
	Belief to Flip: ...
	[Edit] [Expand]
You: "Now generate a LinkedIn hook"	COPY DRAFT
	Hook Score: 8.2/10
	"You have 17 tabs..."

[Type here...]	
----------------	--

- Chat on the left, generated artifacts (Avatars, Copy Drafts, Conversion Chains) materialize on the right.
- You can click into any artifact to expand it into a full editor.
- The chat thread becomes a **session log** of your engineering decisions.
- This is exactly how Claude Artifacts and Cursor Composer work.

☑ Strengths:

Most natural AI interaction model. Users can work in natural language. The system feels alive and responsive. Session history provides context. Scales to any number of artifact types. Matches the mental model of "talking to an AI assistant."

⚠ Weaknesses:

Requires a functioning AI backend (not just UI mockup). Can feel disorienting if artifacts pile up. Less spatial control than a canvas. Harder to visualize the full system architecture at a glance. Users who prefer direct manipulation may find chat-first frustrating.

Best for: When the AI backend is ready and you want users to interact with the system conversationally. This is the most "AI-native" approach — it's how Cursor, Claude, and v0 handle complex tool interaction.

Approach 4: Graph-First Workstation

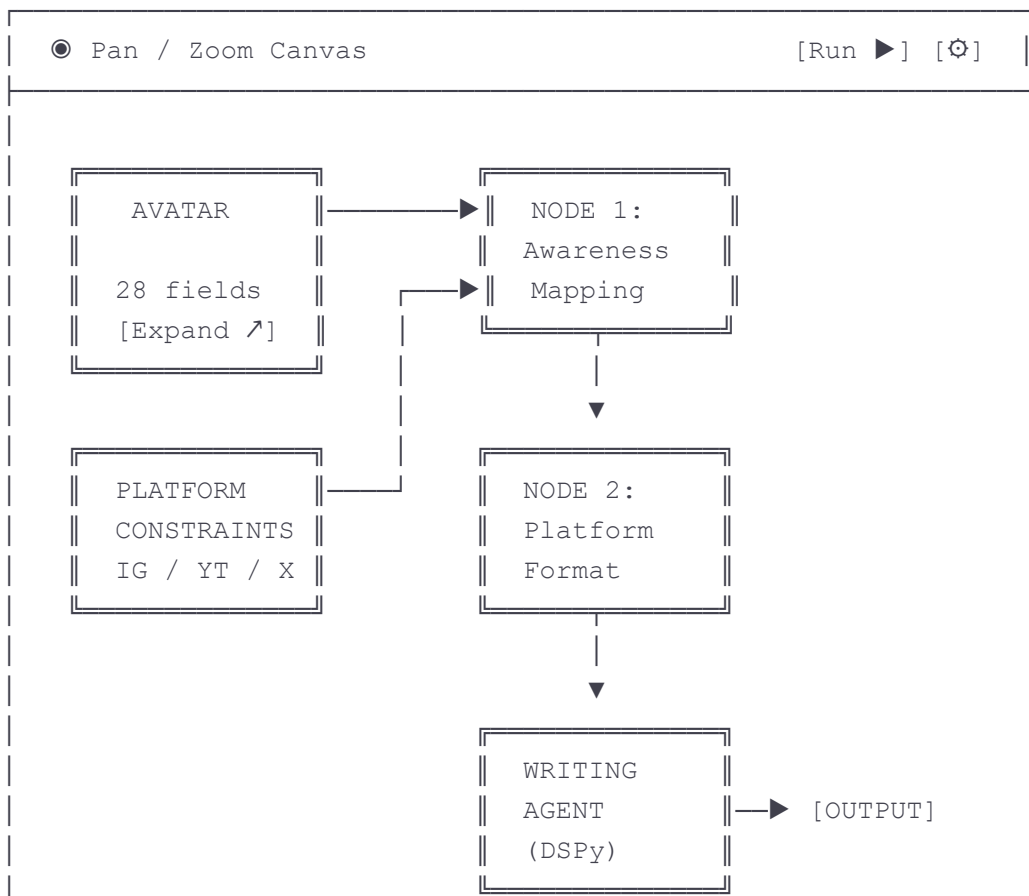
The Unreal / ComfyUI Model

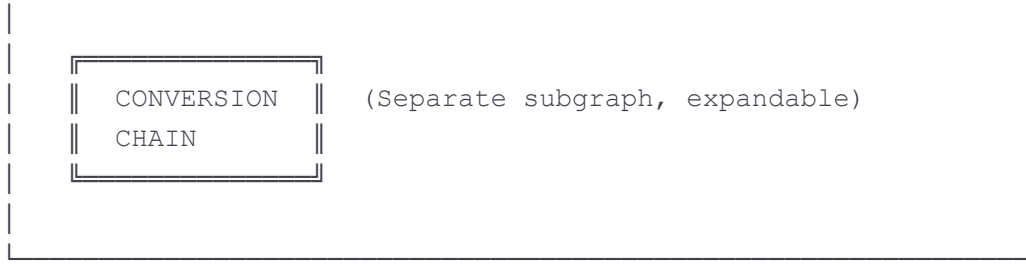
Most Powerful

Inspired by: Unreal Engine Blueprints, ComfyUI, LangFlow, Flowise, n8n

Concept: The entire system is a visual graph. Avatar, Content, and Conversion Chain are not separate pages — they are nodes on a single infinite canvas, connected by data wires.

How it maps to IA:





- Every component (Avatar, Platform, Writing Agent, Judge, Conversion Chain) is a **node on a canvas**.
- Data flows visually via wires (e.g., Avatar psychographics → Awareness Node → Platform Node → Writing Agent).
- Double-clicking any node opens its internal editor (the 28 fields, the copy draft, etc.).
- Subgraphs allow you to collapse complex chains into single nodes for cleanliness.
- This maps directly to your LangGraph architecture: `OnboardingNode → ResearchNode → WritingNode → JudgeNode`.

☑ Strengths:

Directly mirrors the actual system architecture. Users can see and understand the data flow. Most extensible — adding new nodes, agents, or processing steps is trivial. Power users will love it. This is exactly how ComfyUI, LangFlow, and Unreal Engine handle complex pipelines. **Your system IS a graph (LangGraph). The UI should reflect that.**

⚠ Weaknesses:

Highest implementation complexity. Requires a library like React Flow. "Spaghetti" risk if too many nodes. Intimidating for first-time users. Needs strong onboarding/guided mode. Not mobile-friendly.

Best for: The PEW (Psychological Engineering Workstation) mode specifically. This IS the "under the hood" view. It's ComfyUI for marketing. It's the view that makes the system transparent and extensible.

Approach 5: The Hybrid Architecture

The Recommended Build

★ Recommendation

Inspired by: The relationship between Claude Chat ↔ Claude Artifacts, ComfyUI Backend ↔ ComfyUI App Mode, Xcode Interface Builder ↔ Xcode Code Editor

Concept: Don't pick one approach. Layer them. Each mode (Chat / CEA / PEW) gets the interaction model that best suits its purpose.

Mode	Interaction Model	Rationale
Chat	Approach 3 (Conversational Shell)	Natural language interaction. AI generates artifacts. Session log for decisions.
CEA	Approach 2 (Canvas + Inspector)	3 pillars visible on canvas. Click to inspect/edit. Clean, visual, non-intimidating.
PEW	Approach 4 (Graph-First)	Full node canvas. Mirrors LangGraph architecture. Maximum control and transparency.

This creates a natural **progressive disclosure ladder**:



	[Session Log]	[3 Pillars]	[Full Engine]
AUDIENCE:	New Users	Active Users	Power Users
	Onboarding	Daily Workflow	System Design

Why this works:

It respects the user's journey. A new user starts in Chat, talks to the system, gets comfortable. They graduate to CEA when they want visual control. They unlock PEW when they want to see and modify the underlying graph. This is exactly how Apple structures Logic Pro (simple mode → advanced mode) and how ComfyUI has "App Mode" (simplified) vs "Graph Mode" (full power).

Implementation phasing:

1. **Phase 1 (Now):** Build CEA as Canvas + Inspector. This is the highest-leverage UI for daily use.
2. **Phase 2:** Build Chat mode with artifact panels (once AI backend exists).
3. **Phase 3:** Build PEW graph mode with React Flow (once the full LangGraph pipeline is running).

Comparison Matrix

Criterion	1. Drill-Down	2. Canvas+Inspector	3. Chat+Panels	4. Graph-First	5. Hybrid
Cognitive Load	★★★★★	★★★★★	★★★★★	★★	★★★★★
System Awareness	★★	★★★★★	★★★★	★★★★★★	★★★★★★
Extensibility	★★	★★★★	★★★★	★★★★★★	★★★★★★
Implementation Effort	★★★★★★	★★★★	★★★★	★★	★★
Matches Your Architecture	★★	★★★★	★★★★	★★★★★★	★★★★★★
Premium Feel	★★★	★★★★★	★★★★★	★★★★★	★★★★★★

Design Principles from the Best

Apple: Progressive Disclosure

Apple's professional tools (Logic Pro, Final Cut, Xcode) all share one principle: **the surface layer must feel self-evident**. Complex features are pushed into Inspector panels, contextual menus, or secondary windows that only appear when required. The user's cognitive load is only taxed by the specific task they're focusing on.

Applied to IA: The 3 big blocks (Avatar, Content, Conversion Chain) ARE the self-evident surface. The 28 psychographic fields, the belief architecture, the DSPy nodes — these are the Inspector. They appear only when you drill into a specific pillar.

Linear: Command-First Minimalism

Linear keeps its visible UI sparse and uses a command palette (`Cmd+K`) as the primary navigation tool. Everything is keyboard-accessible. The surface is calm; the power is one shortcut away.

Applied to IA: Consider adding a command palette to IA. `Cmd+K` → "Switch to Avatar", "Generate hook for LinkedIn", "Run quality gate". This would let power users fly without clicking through hierarchies.

Figma: Always-Visible Context

Figma never hides the canvas. You can always see the full design while editing a specific component in the Inspector. This maintains spatial awareness — you know where your edit fits in the larger system.

Applied to IA (CEA mode): The 3 pillars should always be visible as blocks on the canvas, even when you're deep-editing one in the Inspector panel. This maintains the mental model of "I'm editing Avatar, but I can see how it connects to Content and Conversion Chain."

Bret Victor: The Ladder of Abstraction

Bret Victor argues that the best tools let users move freely between abstraction levels — zooming in to see individual data points, zooming out to see the whole system. The UI should not trap you at one level.

Applied to IA: This IS the Hybrid architecture (Approach 5). Chat = highest abstraction. CEA Canvas = medium. PEW Graph = lowest. The user moves between them fluidly.

Key People & Resources

Bret Victor

Tools for Thought · Dynamic Mediums

Former Apple designer. Advocates for "explorable explanations" and malleable interfaces. His concept of the Ladder of Abstraction directly applies to IA's multi-level UI.

worrydream.com · [@waboretum](https://twitter.com/waboretum)

Andy Matuschak

Tools for Thought · Apple (former)

Focuses on reducing cognitive load and creating systems that match mental models. His work on "evergreen notes" and spaced repetition systems is directly relevant to how IA should structure knowledge.

andymatuschak.org · [@andy_matuschak](https://twitter.com/andy_matuschak)

Karri Saarinen

Linear · Co-Founder & CEO

Built the most admired project management tool in tech. Linear's command-palette-first, keyboard-driven UI is the gold standard for complex tools that feel simple.

linear.app · [@karrisaarinen](https://twitter.com/karrisaarinen)

ComfyUI Community

Open Source · Node-Based AI

The leading open-source node-based UI for AI image generation. Their "App Mode" (simple

Rasmus Andersson

Figma · Inter Typeface

Designed Inter (the typeface you're using). His work proves that refined typographic hierarchy reduces perceived complexity in feature-heavy apps.

rsms.me · [@rsms](https://twitter.com/rsms)

Mike Matas

Apple · Facebook (former)

Emphasizes direct manipulation and making UI "disappear." His principle: remove jargon, let users interact directly with content. Pioneer of the "swipe to go back" gesture.

[@mike_matas](https://twitter.com/mike_matas)

Guillermo Rauch

Vercel · v0.dev

v0 is the most relevant precedent for IA's chat-to-artifact UI. Chat input on the left, rendered UI artifact on the right. Directly applicable to IA's Chat mode.

vercel.com · [@rauchg](https://twitter.com/rauchg)

React Flow

Open Source Library

The standard library for building node-based UIs in React. Used by Stripe, LangFlow, and Flowise. This

wrapper) vs "Graph Mode" (full nodes) duality is the exact model for CEA vs PEW.

[GitHub](#)

is what you'd use to build PEW's graph mode.

[reactflow.dev](#) · [GitHub](#)

The Glassmorphism Question

What Works in Production

- **Apple's "Liquid Glass" (iOS/macOS):** Control Center, macOS sidebars. Used sparingly, with solid tints beneath the blur, maintaining legibility.
- **Microsoft Fluent Design "Acrylic":** Windows 11 context menus, Settings. Uses subtle noise textures over the blur to prevent "smearing."
- **Your Mars background:** Works well because the background image is low-frequency (smooth gradients, no high-contrast details), which means the blur doesn't create visual noise.

What Fails

- **Full-screen glassmorphism:** Applying it to the entire page or primary reading areas reduces legibility.
- **Cluttered backgrounds beneath glass:** High-contrast or busy backgrounds create visual noise through the blur.
- **Overuse:** When everything is glass, nothing is glass. It loses its ability to create depth and hierarchy.

Recommendation for IA: Use glassmorphism for **floating elements only** — the navigation pill, modal overlays, the Inspector panel border. Keep the main canvas and editor backgrounds as solid dark surfaces (your current `rgba(20, 20, 25, 0.4)` is good). The Mars background provides the ambient warmth; the glass elements float on top of it. Don't make the glass the primary surface — make it the accent.

Psychological Engineering in the UI Itself

Your system is about psychological engineering. **The UI should practice what it preaches.**

Principle 1: Safe Exploration

From your Avatar_Internal.md — the target user fears shipping imperfect work. The UI should make it psychologically safe to experiment: undo/redo, version history, draft states, "no wrong answers" language in the onboarding flow.

Principle 2: Instant Gratification

From your DSPy architecture — the system can generate copy in seconds. Surface this immediately. When a user fills in Avatar data, show a live preview of how the Stimulus engine would use that data. Real-time feedback loops reduce the gap between input and reward.

Principle 3: Progressive Commitment

From your 28-field psychographic interview — don't dump all 28 fields at once. Group them into 4 categories (Demographics → Psychographics → Beliefs → Behaviors). Show a progress bar (you already designed this in your implementation_plan.md). Let each completed category visually "unlock" the next one.

Principle 4: The Prediction Error

From your writing system — the best hooks induce "prediction error" (expecting one thing, getting another). The UI should do this too: when the AI generates copy, show the incorrect belief first (what the user typed in the Avatar), then reveal how the AI flipped it. This makes the system feel magical and builds trust in the engine.

Principle 5: Identity Alignment

From Section 5 of Avatar_Internal.md — this audience responds to "system thinker" language. The UI should use words like "configure," "calibrate," "architect," "constraints," "harness" rather than "set up," "create," "make." The UI language IS the brand.

Recommendations & Next Steps

The Verdict:

Build the **Hybrid Architecture** (Approach 5), but phase it. Don't try to build all three modes at once.

Phase 1: CEA as Canvas + Inspector (Build Now)

This is the highest-leverage UI for immediate use. It gives you:

- The 3 pillars visible at all times on a canvas
- A collapsible Inspector panel for deep editing
- Relationship lines between pillars showing data flow
- The glassmorphic Mars aesthetic you've designed
- Can be built with vanilla HTML/CSS/JS (no React required yet)

Phase 2: Chat Mode with Artifacts (Build When Backend Exists)

Once the LangGraph + DSPy pipeline is running, the Chat mode becomes meaningful. Users can:

- Talk to the system in natural language
- See generated artifacts (Avatar data, copy drafts) materialize as panels
- Edit artifacts inline
- Maintain a session log of all engineering decisions

Phase 3: PEW Graph Mode (Build When System is Mature)

This is the "Claude Code" equivalent — the raw, transparent view of the entire system as a node graph.

Requires:

- React + React Flow (or similar graph library)
- Mapping of LangGraph nodes to visual nodes
- Subgraph collapsing for complexity management
- This is the moat — no other marketing tool exposes its reasoning graph to the user

The Build Stack

Component	Technology	Why
Phase 1 (CEA)	Vanilla HTML/CSS/JS	Fast. No dependencies. Ship today.
Phase 2 (Chat)	Next.js + Server Actions	SSR for chat streaming. API routes for LangGraph.
Phase 3 (PEW Graph)	React + React Flow	Industry standard for node-based UIs.
Styling	Tailwind CSS or Vanilla CSS	Your choice. Both work.
Backend	Python (LangGraph + DSPy)	Already decided in your implementation_plan.md.

END OF RESEARCH DOCUMENT

Compiled from 40+ sources including internal R&D docs, academic papers, and industry analysis.

